

CombatLedger — Claude Code Project Brief

Version: 1.0

Last Updated: February 2026

Target: Claude Code (Sonnet 4.6 default / Opus 4.6 for architecture sessions)

Table of Contents

1. [Project Overview & Goal](#)
 2. [Why This Exists — The Problem Space](#)
 3. [Architecture Overview](#)
 4. [Data Sources & What We Can Access](#)
 5. [Repository Structure](#)
 6. [Tech Stack & Dependencies](#)
 7. [Phase 1 — Foundation & Core Loop](#)
 8. [Phase 2 — CC, Performance & Presets](#)
 9. [Phase 3 — Trend Intelligence & Distribution](#)
 10. [Data Schemas](#)
 11. [WoW Combat Log Reference](#)
 12. [WoW Addon API Constraints \(12.0+\)](#)
 13. [Spell Metadata Database](#)
 14. [UI/UX Specification](#)
 15. [Testing Strategy](#)
 16. [Coding Standards & Conventions](#)
 17. [Known Risks & Mitigations](#)
-

1. Project Overview & Goal

CombatLedger is a two-part system for World of Warcraft (Midnight, patch 12.0+) that bridges the gap between raw combat log data and in-game visualisation:

1. **Companion App** — a lightweight background process (Tauri/TypeScript) that watches `WoWCombatLog.txt`, parses events, and writes structured data to the WoW SavedVariables directory.

2. **In-Game Addon** — a Lua addon that reads those SavedVariables and renders a post-combat analysis UI panel directly inside WoW.

The core value proposition: Neither Blizzard’s native damage meter nor Details! (the dominant third-party meter) can show players the causal chain leading to a death, which targets were interrupted and by whom, or CC coverage chains with Diminishing Returns visualised. CombatLedger delivers all of this by operating on the combat log file — which is completely unaffected by WoW 12.0’s Secret Values API restrictions — and surfacing the results inside the game after combat ends.

Target users: Mythic+ and raid groups who want actionable post-run analysis without leaving the game to visit WarcraftLogs or Archon.

2. Why This Exists — The Problem Space

WoW 12.0 “Secret Values” API Restrictions

With the Midnight expansion (patch 12.0, January 2026), Blizzard introduced the “Secret Values” system. Combat data is now in a **black box** for in-game addons:

- Addons can **display** combat UI elements (resize bars, recolour frames, reposition elements)
- Addons **cannot** read raw combat values, parse combat events for logic, or make decisions based on combat state
- Addon-to-addon communication in instances (e.g., WeakAura sync groups) is blocked
- Enemy combat log data in PvP is fully hidden

What this killed: WeakAuras (discontinued for Retail), ElvUI, rotation helpers, real-time boss mechanic solvers.

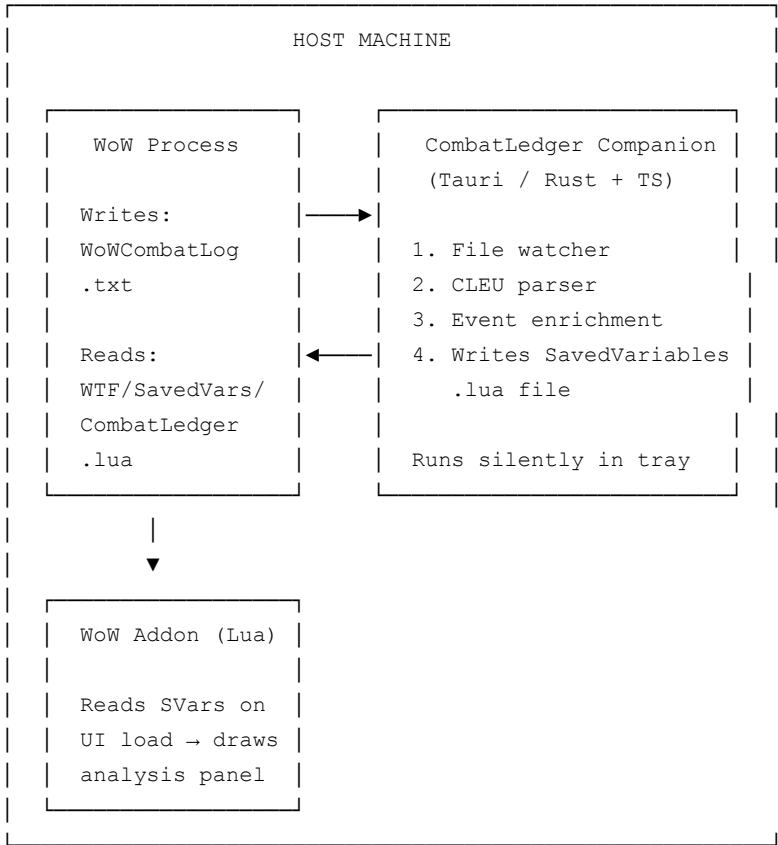
What is unaffected: The `WoWCombatLog.txt` file on disk. This file is written by the game client directly to the filesystem and records every combat event with full fidelity — spell IDs, GUIDs, amounts, timestamps, flags. Secret Values does not touch it. External tools (WarcraftLogs, Archon) have always used this file and continue to work identically.

The Gap CombatLedger Fills

Capability	Blizzard Meter	Details!	WarcraftLogs	CombatLedger
In-game display	✓	✓	✗	✓
Death causal chain	✗	✗	✓ (external)	✓
Interrupt per-target report	✗	✗	✓ (external)	✓
CC coverage with DR	✗	✗	Partial	✓
Defensive usage audit	✗	✗	✓ (external)	✓
No upload required	✓	✓	✗	✓

Post-run trend tracking	✗	✗	✓ (external)	✓ (Phase 3)
-------------------------	---	---	--------------	-------------

3. Architecture Overview



Communication Flow

1. Player runs a Mythic+ key or raid
2. Companion app tails `WoWCombatLog.txt` in real time, buffering events per encounter
3. On detecting `ENCOUNTER_END` (or `ZONE_CHANGED_NEW_AREA`), companion parses the buffered session and writes enriched data to `WTF/Account/<ACCOUNT>/SavedVariables/CombatLedger.lua`
4. Player types `/cl` or `/reload` in-game
5. Addon reads `SavedVariables` on load, renders the post-combat panel

Key Constraint: No Real-Time In-Game Sync

`SavedVariables` are **only read by WoW at UI load time** and **only written by WoW at logout / `/reload`**. This means:

- The addon cannot hot-reload data mid-session
- The natural trigger is a `/reload` between pulls or after a dungeon

- This is an acceptable UX pattern — players routinely `/reload` between keys
- The companion should write data as fast as possible after `ENCOUNTER_END` so it's ready before the player reloads

4. Data Sources & What We Can Access

WoWCombatLog.txt — Full Fidelity, Unrestricted

Located at: `<WoW Install>/Logs/WoWCombatLog.txt`

The companion app reads this file. It is written by the game client continuously during play. Relevant event types:

Event	Data Available
COMBATANT_INFO	Player GUID, spec, talents, gear at pull time
ENCOUNTER_START	Encounter ID, encounter name, difficulty, group size
ENCOUNTER_END	Encounter ID, success/fail
SPELL_DAMAGE	Source GUID, dest GUID, spell ID, amount, overkill, resisted, blocked, absorbed, critical flag
SPELL_PERIODIC_DAMAGE	Same as SPELL_DAMAGE for DoTs
SWING_DAMAGE	Melee hits — source, dest, amount, overkill
RANGE_DAMAGE	Ranged hits
SPELL_HEAL	Source, dest, spell ID, amount, overheal, absorbed, critical
SPELL_PERIODIC_HEAL	HoT ticks
UNIT_DIED	GUID of unit that died, unconscious flag
SPELL_INTERRUPT	Source GUID, dest GUID, spell ID (interrupt spell), extraSpellID (interrupted spell), extraSchool
SPELL_CAST_START	Source GUID, spell ID, target
SPELL_CAST_SUCCESS	Source GUID, spell ID, target
SPELL_CAST_FAILED	Source GUID, spell ID, reason
SPELL_AURA_APPLIED	Source, dest, spell ID, aura type (BUFF/DEBUFF), amount
SPELL_AURA_REMOVED	Source, dest, spell ID, aura type
SPELL_AURA_REFRESH	Source, dest, spell ID

WoW Addon API — Restricted but Available

The in-game addon can access Blizzard's **native damage meter API** (introduced 12.0) for DPS/HPS numbers. This is the only runtime data source. All other data comes from SavedVariables written by the companion.

Relevant permitted APIs for the addon:

- `GetCombatRating()` — player stats
- Frame positioning, sizing, texturing — unrestricted
- `UnitName()` , `UnitClass()` , `UnitGUID()` — for live player identification
- `GetSpecialization()` — player spec
- Blizzard's native meter data hooks (12.0+) — for DPS/HPS display

5. Repository Structure

```

combatledger/
├── companion/                                # Tauri app (Rust + TypeScript)
│   ├── src-tauri/
│   │   ├── src/
│   │   │   ├── main.rs                      # Tauri entry, system tray, window management
│   │   │   ├── watcher.rs                  # File watcher (notify crate)
│   │   │   ├── writer.rs                   # SavedVariables writer
│   │   │   └── paths.rs                    # WoW install path detection
│   │   ├── Cargo.toml
│   │   └── tauri.conf.json
│   └── src/
│       ├── parser/
│       │   ├── index.ts                    # Parser orchestration
│       │   ├── cleu.ts                     # Raw CLEU line parser → typed events
│       │   ├── session.ts                  # Session/encounter segmentation
│       │   ├── death.ts                    # Death recap engine
│       │   ├── interrupt.ts                # Interrupt report engine
│       │   ├── cc.ts                       # CC coverage + DR engine (Phase 2)
│       │   └── defensive.ts                # Defensive audit engine (Phase 2)
│       ├── enrichment/
│       │   ├── spelldb.ts                  # Spell metadata lookup
│       │   └── guid.ts                     # GUID → player name/class resolution
│       ├── serializer/
│       │   └── savedvars.ts                 # TypeScript → Lua table serializer
│       ├── db/
│       │   └── sessions.ts                 # SQLite session history (Phase 3)
│       └── main.ts                         # App entry point
├── data/
│   └── spells.json                         # Spell metadata database
└── package.json

```

```

|
|└─ addon/                                # WoW Lua addon
|   |└─ CombatLedger.toc                  # Addon table of contents
|   |└─ CombatLedger.lua                  # Entry point, slash commands, minimap button
|   |└─ core/
|   |   |└─ Frame.lua                     # Main addon frame, tab system
|   |   |└─ Data.lua                     # SavedVariables reader + data access layer
|   |   |└─ Config.lua                   # Settings, presets (AceConfig)
|   |└─ tabs/
|   |   |└─ Overview.lua                 # Overview tab
|   |   |└─ Deaths.lua                  # Death recap tab
|   |   |└─ Interrupts.lua              # Interrupt report tab
|   |   |└─ CC.lua                      # CC coverage tab (Phase 2)
|   |   |└─ Performance.lua             # Performance tab (Phase 2)
|   |└─ libs/
|   |   |└─ Ace3/                        # Ace3 framework
|   |   |└─ LibSharedMedia-3.0/        # Shared textures/fonts
|
|└─ data/
|   |└─ spells/
|   |   |└─ schema.md                   # Spell DB schema documentation
|   |   |└─ base.json                   # Core spells (shared across dungeons)
|   |   |└─ dungeons/
|   |   |   |└─ darkflame-cleft.json
|   |   |   |└─ priory-sacred-flame.json
|   |   |   |└─ ...
|
|└─ tests/
|   |└─ fixtures/                       # Sample WoWCombatLog.txt excerpts
|   |└─ parser/                         # Parser unit tests
|   |└─ integration/                   # Full pipeline tests
|
|└─ COMBATLEDGER_CLAUDE_BRIEF.md        # This file

```

6. Tech Stack & Dependencies

Companion App

Technology	Version	Purpose
Tauri	2.x	App shell, system tray, native packaging
Rust	stable	Tauri backend, file watcher (notify crate), SavedVariables writer
TypeScript	5.x	Parser logic, enrichment, serialisation
Node.js	20+	TypeScript runtime within Tauri
notify (Rust crate)	6.x	Cross-platform file watcher
rusqlite	0.31+	SQLite bindings for Phase 3 session history

Vitest	latest	Unit testing for TypeScript
--------	--------	-----------------------------

In-Game Addon

Technology	Version	Purpose
Lua	5.1 (WoW runtime)	All addon code
Ace3	latest	Addon framework (AceAddon, AceDB, AceConfig, AceGUI)
LibSharedMedia-3.0	latest	Shared textures and fonts

Key Rust Crates

```
[dependencies]
tauri = { version = "2", features = ["tray-icon", "system-tray"] }
notify = "6"
rusqlite = { version = "0.31", features = ["bundled"] } # Phase 3
serde = { version = "1", features = ["derive"] }
serde_json = "1"
tokio = { version = "1", features = ["full"] }
```

7. Phase 1 — Foundation & Core Loop

Duration: ~8 weeks

Milestone: Alpha release — Deaths tab and Interrupts tab functional

Model recommendation: Sonnet 4.6 for implementation; Opus 4.6 for SavedVariables schema design session before starting

Phase 1 Tasks (in order)

Task 1.1 — Tauri App Scaffold

Build the companion app shell:

- System tray icon with menu: "Open", "Settings", "Quit"
- Auto-detect WoW installation path:
 - Windows: check registry `HKEY_LOCAL_MACHINE\SOFTWARE\Blizzard Entertainment\World of Warcraft`, fallback to `C:\Program Files (x86)\World of Warcraft\_retail_`
 - macOS: check `/Applications/World of Warcraft/_retail_`
 - Expose a settings screen if auto-detect fails, with a folder picker
- Derive paths:
 - `<wow_path>/Logs/WoWCombatLog.txt` — combat log

- `<wow_path>/WTF/Account/<ACCOUNT>/SavedVariables/CombatLedger.lua` — output (note: account name must be discovered, see Task 1.2)

- Show connection status in tray: “WoW log found ✓” or “Log not found”

Acceptance criteria: App launches, detects WoW path, shows tray icon, does not crash when WoW is not running.

Task 1.2 — WTF Path Discovery

The SavedVariables path includes the Battle.net account name, which varies per user. Strategy:

1. List directories under `<wow_path>/WTF/Account/`
2. Filter out directories that start with `#` (these are internal Blizzard directories)
3. If exactly one account directory exists, use it
4. If multiple exist, list them in settings for the user to select
5. Construct full path: `<wow_path>/WTF/Account/<ACCOUNT>/SavedVariables/CombatLedger.lua`

Acceptance criteria: Path is correctly resolved for single-account and multi-account WoW installations.

Task 1.3 — File Watcher

Implement a file watcher on `WoWCombatLog.txt`:

- Use Rust `notify` crate with `RecommendedWatcher`
- Watch for `Modify` events (WoW appends to the file)
- On event: read only the **new bytes** since last read position (tail behaviour — do not re-read the entire file)
- Send new lines to the TypeScript parser via Tauri invoke/event bridge
- Handle file rotation: if file size is smaller than last known position, WoW has rotated the log — reset position to 0
- Handle the case where the log doesn't exist yet (WoW hasn't been launched)

Acceptance criteria: Watcher correctly tails the log file. New events are delivered to the parser within 500ms of being written.

Task 1.4 — CLEU Line Parser

Parse raw combat log lines into typed TypeScript structs. Each line format:

```
MM/DD HH:MM:SS.mmm CLEU_EVENT_TYPE,param1,param2,...
```

First 8 parameters are always present (the “base” params):

1. `timestamp` — wall clock time

2. `subevent` — the event type string (e.g., `SPELL_DAMAGE`)
3. `hideCaster` — boolean
4. `sourceGUID`
5. `sourceName`
6. `sourceFlags`
7. `sourceRaidFlags`
8. `destGUID`
9. `destName`
10. `destFlags`
11. `destRaidFlags`

After these, parameters vary by event type. Implement parsers for all event types listed in Section 4.

Typed event interface pattern:

```
interface BaseEvent {
    timestamp: Date;
    subevent: string;
    sourceGUID: string;
    sourceName: string;
    destGUID: string;
    destName: string;
}

interface SpellDamageEvent extends BaseEvent {
    subevent: 'SPELL_DAMAGE' | 'SPELL_PERIODIC_DAMAGE';
    spellId: number;
    spellName: string;
    spellSchool: number;
    amount: number;
    overkill: number; // -1 if not a killing blow, positive value = overkill damage
    resisted: number;
    blocked: number;
    absorbed: number;
    critical: boolean;
}

// ... etc for each event type
```

Acceptance criteria: Parser correctly handles all event types in the test fixtures. No crashes on malformed lines (log with garbage data). Test fixture: a real 1000-line excerpt from a Mythic+ run.

Task 1.5 — Session Segmentation

Group parsed events into encounters:

```

interface EncounterSession {
    encounterId: number;
    encounterName: string;
    difficulty: number;
    groupSize: number;
    startTime: Date;
    endTime: Date;
    success: boolean;
    pullNumber: number; // 1-indexed within the zone session
    events: ParsedEvent[];
    combatants: CombatantInfo[]; // from COMBATANT_INFO events
}

```

Logic:

- `ENCOUNTER_START` → create new `EncounterSession`, begin buffering events
- `ENCOUNTER_END` → close session, trigger analysis pipeline
- If `ENCOUNTER_START` fires without a preceding `ENCOUNTER_END`, close the previous session (wipe scenario)
- Track pull number: reset to 1 on `ZONE_CHANGED_NEW_AREA`, increment on each `ENCOUNTER_START`
- Keep last N sessions in memory (configurable, default 10)

Acceptance criteria: A Mythic+ run with 3 pulls correctly produces 3 `EncounterSession` objects with accurate events lists.

Task 1.6 — Death Recap Engine

For each `UNIT_DIED` event where the dead unit is a player (check `destFlags` for `COMBATLOG_OBJECT_TYPE_PLAYER`):

1. Look back through the encounter events for the last **10 seconds** before the death timestamp
2. Collect all damage events targeting that player GUID, sorted chronologically
3. Collect all heal events targeting that player GUID in the same window
4. Identify the fatal event: the last damage event where `overkill > 0` OR the event that pushed the player to 0 HP
5. Calculate running HP estimate: start from last known max HP (from `COMBATANT_INFO`), apply damage/heal deltas in order. Flag events where HP dropped below 30% (critical threshold) and below 0% (death)
6. For the fatal event, note the `overkill` value (how much excess damage)
7. Cross-reference known defensive spell IDs for the dead player's class/spec — check `SPELL_CAST_SUCCESS` events for those spell IDs in the 30 seconds before death. Flag defensives that were used (and when) and defensives that were available but unused (CD not on cooldown at time of death)

Output structure:

```
interface DeathRecap {
  playerGUID: string;
  playerName: string;
  playerClass: string;
  playerSpec: string;
  deathTimestamp: Date;
  pullNumber: number;
  timeIntoPull: number; // seconds since ENCOUNTER_START
  fatalSpellId: number;
  fatalSpellName: string;
  fatalSourceName: string;
  overkill: number;
  events: DeathEvent[]; // last 10s of damage/heal events
  availableDefensives: DefensiveStatus[];
}

interface DeathEvent {
  timestamp: Date;
  type: 'damage' | 'heal';
  sourceGUID: string;
  sourceName: string;
  spellId: number;
  spellName: string;
  amount: number;
  isFatal: boolean;
  estimatedHpAfter: number; // percentage, 0-100
}

interface DefensiveStatus {
  spellId: number;
  spellName: string;
  wasUsed: boolean;
  usedAt?: Date; // if wasUsed
  available: boolean; // true if CD was not on cooldown at time of death
}
```

Acceptance criteria: Death recap for a test scenario with a known fatal sequence matches manual log analysis exactly.

Task 1.7 — Interrupt Report Engine

For each interruptible enemy cast in an encounter:

1. Collect all `SPELL_CAST_START` events where source is an enemy (check `sourceFlags`)
2. Cross-reference the spell ID against the spell metadata database — only process spells flagged `interruptible: true`
3. For each interruptible cast, find the corresponding `SPELL_INTERRUPT` event (match on `destGUID + extraSpellID` = the interrupted spell)

4. If interrupted: record who interrupted (sourceGUID/Name), which interrupt spell was used (spellId), and what % of cast time had elapsed at interrupt (requires tracking `SPELL_CAST_START` timestamp vs `SPELL_INTERRUPT` timestamp vs known cast duration from spell DB)
5. If NOT interrupted (no matching `SPELL_INTERRUPT` before the next `SPELL_CAST_SUCCESS` for that spell): flag as missed, note reason if determinable (e.g., "no player interrupt spells cast in ± 1 s window")

Output structure:

```
interface InterruptReport {
  encounterId: number;
  interrupts: InterruptEvent[];
  summary: {
    total: number;
    intercepted: number;
    missed: number;
    ratePercent: number;
  };
  byPlayer: Record<string, {
    playerName: string;
    intercepted: number;
    opportunities: number; // times they were in range when a kick was needed
  }>;
}

interface InterruptEvent {
  timestamp: Date;
  pullNumber: number;
  targetName: string; // enemy being interrupted
  targetGUID: string;
  spellId: number; // spell being cast by enemy
  spellName: string;
  castDurationMs: number; // from spell DB
  wasInterrupted: boolean;
  interruptedBy?: string; // player name
  interruptSpellId?: number;
  castPercentAtInterrupt?: number; // 0-100
  missedReason?: string; // "no interrupter", "all CDs on cooldown", "cast completed"
}
```

Acceptance criteria: Interrupt report matches manual count from test log. Missed casts are correctly identified.

Task 1.8 — SavedVariables Serialiser

Convert the parsed TypeScript data structures into a valid Lua table file:

```
-- CombatLedger.lua (SavedVariables)
CombatLedgerDB = {
  ["version"] = 1,
  ["sessions"] = {
    [1] = {
      ["encounterId"] = 2660,
```

```

["encounterName"] = "Darkflame Cleft",
["difficulty"] = 8,
["startTime"] = 1737900000, -- Unix timestamp
["endTime"] = 1737900278,
["success"] = true,
["deaths"] = {
    [1] = {
        ["playerName"] = "Valdris",
        ["playerClass"] = "PALADIN",
        -- ... etc
    }
},
["interrupts"] = { ... },
}
}

```

Rules:

- All strings must be properly escaped for Lua
- Numbers are unquoted
- Booleans are `true / false`
- Tables use `[index]` for array entries
- Timestamps stored as Unix epoch integers
- Maximum file size target: 500KB per session, warn if exceeded, trim oldest events first

Acceptance criteria: Written file can be loaded by WoW without errors. `loadfile()` on the output produces a valid Lua table.

Task 1.9 — Addon Core Shell

Build the WoW Lua addon:

- `CombatLedger.toc` with correct interface version for Midnight (120002 for 12.0.2, update as needed)
- Register with AceAddon-3.0
- AceDB-3.0 for SavedVariables (this is separate from our companion-written data — AceDB manages addon settings; companion writes to a separate `CombatLedgerDB` global)
- Slash command `/cl` — opens main frame
- Slash command `/cl reset` — clears stored data
- Minimap button (LibDBIcon or similar)
- Main frame: 960x600 pixels, movable, closeable, remembers position
- Tab bar: Overview, Deaths, Interrupts (Phase 1); CC, Performance (Phase 2)
- Filter bar: player toggle chips, pull filter chips, preset dropdown

- All tabs read from `CombatLedgerDB` global (written by companion)
- Show a “No data — run a dungeon then /reload” placeholder if `CombatLedgerDB` is nil or empty

Acceptance criteria: Addon loads without Lua errors. Frame opens/closes. Tabs switch. Empty state is handled gracefully.

Task 1.10 — Deaths Tab UI

Render the death recap data:

- One card per death event, ordered chronologically
- Card header: player class colour, spec abbreviation, player name, role, pull number, time-into-pull
- Expandable: click header to expand/collapse the timeline
- Timeline: list of events (damage/heal) in the 10s window, rendered as rows
 - Columns: time-offset, icon (damage/heal/fatal), source name, spell name, amount
 - Fatal event highlighted with red border and skull icon
 - Overkill shown on fatal row
 - Running HP bar (3px horizontal bar above event list showing HP trajectory)
- Defensive note: below timeline, plain-language note if an available defensive was unused
- Empty state: “No deaths recorded in this session”

Task 1.11 — Interrupts Tab UI

Render the interrupt report:

- Header row showing total interrupted / total opportunities / rate %
- Table with columns: Target · Cast, Interrupted By, Spell Used, Cast Progress, Status
- Cast Progress: small progress bar showing % of cast elapsed at interrupt (green) or full red bar if missed
- Status badge: “✓ Kicked” (blue), “× Missed” (red), or “~ Partial” (amber)
- Rows sorted by pull number, then chronologically
- Missed rows have subtle red background tint
- Summary bar at bottom: per-player interrupt count

8. Phase 2 — CC, Performance & Presets

Duration: ~10 weeks (weeks 9–18)

Milestone: Beta release — all 5 tabs functional

Model recommendation: Opus 4.6 for DR tracking system design; Sonnet 4.6 for implementation

Phase 2 Tasks

Task 2.1 — CC Coverage Engine with DR Tracking

Diminishing Returns rules (WoW 12.0):

- DR categories are groups of spells that share a DR timer per target
- Each application of a DR category to a target reduces effectiveness: 100% → 50% → 25% → immune
- DR timer: 18 seconds after a CC expires with no reapplication, DR resets
- In PvP (12.0 change): immune after 2nd application (was 3rd)
- Stun DR applies to: stuns, knockbacks (certain)
- Incapacitate DR: Polymorph, Hex, Sap, etc.
- Sleep DR: Hibernate, etc.
- See `data/spells/base.json` for DR category assignments

Engine logic:

```
interface DRState {
  targetGUID: string;
  categoryStates: Record<string, {
    applications: number;    // 0, 1, 2, 3+ (immune)
    lastExpiry: Date | null; // when last CC expired
    resetAt: Date | null;    // when DR resets (lastExpiry + 18s)
  }>;
}
```

For each `SPELL_AURA_APPLIED` where the spell is in a CC category:

1. Look up DR state for targetGUID + CC category
2. If `resetAt` is past (or null), reset applications to 0
3. Calculate effective duration: `baseDuration * (1 / 2^applications)`
4. Increment applications
5. Store CC segment: who applied, what spell, when, effective duration, DR multiplier

For each `SPELL_AURA_REMOVED`:

1. Record actual duration (end - start timestamp)
2. Update DR state lastExpiry
3. Flag if CC broke early (actual duration < effective duration by >0.5s) and reason if known (damage, spell)

Output structure:

```
interface CCCoverage {
  targetGUID: string;
```

```

    targetName: string;
    targetType: string; // from spell DB / unit flags
    segments: CCSegment[];
    totalCombatDuration: number; // seconds enemy was in combat
    coveredDuration: number; // seconds effectively CC'd
    coveragePercent: number;
}

interface CCSegment {
    sourceGUID: string;
    sourceName: string;
    spellId: number;
    spellName: string;
    ccCategory: string;
    startTime: Date;
    endTime: Date;
    theoreticalDuration: number;
    actualDuration: number;
    drMultiplier: number; // 1.0, 0.5, 0.25, 0.0 (immune)
    brokeEarly: boolean;
    brokeReason?: string;
}

```

Task 2.2 — CC Coverage Tab UI

- One row per CC'd target
- Each row: target name + type, horizontal timeline bar, coverage % badge
- Timeline bar: coloured segments by player (player class colour), opacity indicating DR level
- Gap segments shown between CC windows
- Segments interactive: hover shows tooltip (player, spell, duration, DR multiplier)
- Legend below: player colour → name mapping, DR opacity guide
- Filter: show only targets with < 50% coverage (problem targets)

Task 2.3 — Performance Tab

Data sources:

- DPS/HPS: from Blizzard's native meter API (accessed in addon) — store in SavedVariables alongside companion data when addon syncs
- Cast uptime: calculated from log — time in `SPELL_CAST_SUCCESS` events vs encounter duration
- Interrupts: from interrupt engine
- CC contributed: count of CC segments where sourceGUID = this player
- Deaths: from death engine
- Defensive usage: from defensive audit engine (Task 2.4)

One row per player, sorted by role (tank, healer, DPS):

- Player avatar (class colour, spec abbreviation)
- DPS or HPS bar (normalised to group maximum)
- Uptime % bar
- Stats cluster: interrupts, CC count, deaths, key defensive casts

Task 2.4 — Defensive Usage Audit

Maintain a database of class defensive spells per spec:

```
{
  "PALADIN": {
    "RETRIBUTION": {
      "defensives": [
        { "spellId": 642, "name": "Divine Shield", "durationSec": 8, "cooldownSec": 300 },
        { "spellId": 184662, "name": "Shield of Vengeance", "durationSec": 15, "cooldownSec": 12 },
        { "spellId": 1022, "name": "Blessing of Protection", "durationSec": 10, "cooldownSec": 10 }
      ]
    }
  }
}
```

For each death recap:

1. At time of death, determine which defensives were on cooldown vs available
2. Track `SPELL_CAST_SUCCESS` events for each defensive spell ID for that player throughout the encounter
3. Calculate last-used time for each defensive; determine if CD had expired by time of death
4. Flag: "Divine Shield was available (not used in 5+ minutes)" or "Divine Shield was on cooldown (used 43s ago, 257s remaining)"

Task 2.5 — Preset System

In-game addon settings:

```
-- Preset schema
{
  name = "M+ Push Review",
  filters = {
    players = { "all" }, -- or specific player names
    pulls = "all", -- or "boss", "1", "2", etc.
    activeTab = "deaths",
    showOnlyAvoidable = false,
  }
}
```

Built-in presets (read-only):

- “Default” — all players, all pulls, Overview tab
- “Deaths Only” — all players, all pulls, Deaths tab, deaths expanded
- “Tank Focus” — tank player only, Performance tab
- “Healer Audit” — healer only, Performance tab, overhealing visible
- “M+ Push Review” — all players, Deaths + Interrupts tabs

User can save custom presets. Stored in AceDB SavedVariables (addon settings, not companion data).

Task 2.6 — Export Functionality

From the companion app (system tray menu → “Export last session”):

- CSV export: one file per tab’s data (deaths.csv, interrupts.csv, etc.)
- JSON export: full session data in schema-documented format
- Save to user’s Downloads folder by default, configurable

9. Phase 3 — Trend Intelligence & Distribution

Duration: ~10 weeks (weeks 19–28)

Milestone: v1.0 stable release

Model recommendation: Opus 4.6 for SQLite schema design and trend normalisation logic; Sonnet 4.6 for implementation

Phase 3 Tasks

Task 3.1 — SQLite Session History

Add persistent session storage to the companion app:

```
CREATE TABLE sessions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  encounter_id INTEGER NOT NULL,
  encounter_name TEXT NOT NULL,
  difficulty INTEGER NOT NULL,
  zone_name TEXT,
  key_level INTEGER, -- Mythic+ key level, NULL for raids
  started_at INTEGER NOT NULL, -- Unix timestamp
  ended_at INTEGER NOT NULL,
  success INTEGER NOT NULL, -- 0 or 1
  group_composition TEXT -- JSON array of {name, class, spec}
);

CREATE TABLE session_deaths (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id INTEGER REFERENCES sessions(id),
  player_name TEXT NOT NULL,
```

```

    player_class TEXT NOT NULL,
    player_spec TEXT NOT NULL,
    pull_number INTEGER,
    time_into_pull_ms INTEGER,
    fatal_spell_id INTEGER,
    fatal_spell_name TEXT,
    had_unused_defensive INTEGER -- 0 or 1
);

CREATE TABLE session_interrupts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id INTEGER REFERENCES sessions(id),
    total_opportunities INTEGER,
    total_intercepted INTEGER,
    missed INTEGER
);

CREATE TABLE session_player_stats (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id INTEGER REFERENCES sessions(id),
    player_name TEXT NOT NULL,
    player_class TEXT NOT NULL,
    player_spec TEXT NOT NULL,
    interrupts_attempted INTEGER,
    interrupts_landed INTEGER,
    cc_count INTEGER,
    deaths INTEGER,
    uptime_percent REAL
);

```

Task 3.2 — Trend Engine

Calculate trend metrics across sessions for the same dungeon + difficulty + group composition (fuzzy match: at least 3/5 same players):

```

interface PlayerTrend {
    playerName: string;
    metric: 'interrupt_rate' | 'deaths' | 'uptime' | 'cc_count';
    sessions: Array<{
        sessionId: number;
        date: Date;
        value: number;
    }>;
    trend: 'improving' | 'declining' | 'stable';
    trendPercent: number; // % change over last 5 sessions
}

```

Write trend data to SavedVariables alongside session data so the addon can display it.

Task 3.3 — Trend Tab UI

- One row per player per metric

- Spark-line style mini chart (5-run history) rendered as a series of dots + connecting lines using WoW's texture drawing APIs
- Trend arrow: ↑ green (improving), ↓ red (declining), → grey (stable)
- "Last 5 runs" label
- Click to expand full session list

Task 3.4 — Companion One-Click Installer

Build platform installers:

- Windows: NSIS installer bundled with Tauri's Windows target
- macOS: standard `.dmg` with drag-to-Applications
- Auto-launch on system start: opt-in checkbox during install
- CurseForge: publish as a "Premium Addon" bundle (addon ZIP + companion installer link)
- GitHub Releases: primary distribution for companion app
- Addon on CurseForge: standard addon listing pointing users to GitHub for companion

Task 3.5 — Raid Encounter Support

Extend spell metadata to cover:

- Current Midnight Season 1 raid (encounter IDs when announced)
- Raid-specific interrupt priorities
- Raid defensive audit (more complex: external defensives matter — e.g., Blessing of Protection cast on another player)

10. Data Schemas

SavedVariables Top-Level Structure

```
CombatLedgerDB = {
  version = 2,
  generatedAt = 1737900000, -- Unix timestamp of last companion write
  companionVersion = "0.1.0",
  sessions = {
    -- Array of SessionData tables, newest first
    -- Max 20 sessions retained (trim oldest)
  },
  trends = {
    -- Phase 3 only
    -- playerName → metric → array of {sessionIndex, value}
  }
}
```

SessionData

```
{
  encounterId = 2660,
  encounterName = "Darkflame Cleft",
  difficulty = 8,          -- 8 = Mythic, 23 = Mythic+
  keyLevel = 14,          -- nil for non-M+
  startTime = 1737900000,
  endTime = 1737900278,
  durationSec = 278,
  success = true,
  pullNumber = 1,         -- 1-indexed within zone session
  combatants = {
    { guid = "Player-...", name = "Valdris", class = "PALADIN", spec = "RETRIBUTION" },
    -- ...
  },
  deaths = { ... },       -- Array of DeathRecap
  interrupts = { ... },   -- InterruptReport
  ccCoverage = { ... },   -- Array of CCCoverage (Phase 2)
  playerStats = { ... },  -- Array of PlayerStats (Phase 2)
}
```

11. WoW Combat Log Reference

Log Line Format

```
8/17 20:14:52.441 SPELL_DAMAGE,Player-1234-ABCD1234,"Valdris",0x511,0x0,Creature-0-1234-2660-12
```

Fields: `TIMESTAMP`

`EVENT_TYPE,sourceGUID,sourceName,sourceFlags,sourceRaidFlags,destGUID,destName,destFlags,destRaidFlags,`
`[event-specific...]`

GUID Format

- **Players:** `Player-REALM_ID-CHARACTER_ID` **e.g.** `Player-1234-ABCD1234`
- **Creatures:** `Creature-0-MAP-ZONE-INSTANCE-NPC_ID-SPAWN_UID`
- Check `sourceFlags` bit field for type:
 - `0x0400` = `COMBATLOG_OBJECT_TYPE_PLAYER`
 - `0x0010` = `COMBATLOG_OBJECT_CONTROL_PLAYER` (player controlled)
 - `0x0040` = `COMBATLOG_OBJECT_REACTION_HOSTILE`

Unit Flags Reference

```

const UNIT_FLAGS = {
    AFFILIATION_MINE:      0x00000001,
    AFFILIATION_PARTY:     0x00000002,
    AFFILIATION_RAID:      0x00000004,
    AFFILIATION_OUTSIDER:  0x00000008,
    REACTION_FRIENDLY:     0x00000010,
    REACTION_NEUTRAL:      0x00000020,
    REACTION_HOSTILE:      0x00000040,
    CONTROL_PLAYER:        0x00000100,
    CONTROL_NPC:           0x00000200,
    TYPE_PLAYER:           0x00000400,
    TYPE_NPC:              0x00000800,
    TYPE_PET:              0x00001000,
    TYPE_GUARDIAN:         0x00002000,
    TYPE_OBJECT:           0x00004000,
} as const;

function isPlayer(flags: number): boolean {
    return (flags & UNIT_FLAGS.TYPE_PLAYER) !== 0;
}

function isEnemy(flags: number): boolean {
    return (flags & UNIT_FLAGS.REACTION_HOSTILE) !== 0;
}

```

COMBATANT_INFO Parameters

Emitted at the start of each encounter for each group member:

```

COMBATANT_INFO, playerGUID, faction, strength, agility, stamina, intellect, dodge, parry, block,
critMelee, critRanged, critSpell, speed, lifesteal, hasteMelee, hasteRanged, hasteSpell,
avoidance, mastery, versatilityDamageDone, versatilityHealingDone, versatilityDamageTaken,
armor, currentSpecID, [stats...], [equipment...], [interestingAuras...], [talentInfo...]

```

Key fields: `playerGUID` (index 0), `currentSpecID` (for spec identification).

12. WoW Addon API Constraints (12.0+)

What the Addon CAN Do

- Read `CombatLedgeDB SavedVariables` (our companion-written data) — this is standard Lua global access
- Access Blizzard's native damage meter API for DPS/HPS numbers
- `UnitName("player")`, `UnitClass("player")`, `GetSpecialization()` — own player info
- `UnitGUID("party1")` etc. — party member GUIDs for cross-referencing
- All frame creation, positioning, texturing, font operations

- `C_Timer.After()` , `C_Timer.NewTicker()` — timers for UI animations
- `LibSharedMedia` textures
- `AceDB`, `AceConfig`, `AceGUI`

What the Addon CANNOT Do (12.0 Secret Values)

- Parse `CombatLogGetCurrentEventInfo()` for combat logic (visual display only, restricted)
- Access enemy health values, cooldowns, cast progress programmatically
- Communicate with other addons in instances via private channels
- Make network requests of any kind
- Read files from the filesystem

Critical Addon Rule

The addon is purely a reader and renderer. All data processing happens in the companion app. If a data transformation seems like it should happen in Lua, move it to TypeScript instead.

TOC File Template

```
## Interface: 120002
## Title: CombatLedger
## Notes: Post-combat analysis — requires CombatLedger Companion app
## Version: 0.1.0
## Author: [your name]
## SavedVariables: CombatLedgerSettings
## SavedVariablesPerCharacter: CombatLedgerCharSettings
## X-Companion-Required: true

libs\Ace3\AceAddon-3.0\AceAddon-3.0.lua
libs\Ace3\AceDB-3.0\AceDB-3.0.lua
libs\Ace3\AceConfig-3.0\AceConfig-3.0.lua
libs\Ace3\AceGUI-3.0\AceGUI-3.0.lua
libs\Ace3\AceConsole-3.0\AceConsole-3.0.lua

core\Data.lua
core\Config.lua
core\Frame.lua
tabs\Overview.lua
tabs\Deaths.lua
tabs\Interrupts.lua
tabs\CC.lua
tabs\Performance.lua
CombatLedger.lua
```

Note: `CombatLedgerDB` (companion data) is NOT in `## SavedVariables` — it is written directly by the companion and read as a raw global. `CombatLedgerSettings` (addon settings/presets) IS managed by `AceDB`.

13. Spell Metadata Database

Schema (data/spells/base.json)

```
{
  "spells": {
    "6552": {
      "name": "Pummel",
      "class": "WARRIOR",
      "type": "interrupt",
      "interruptible": false,
      "cooldownSec": 15,
      "notes": "Warrior interrupt"
    },
    "2139": {
      "name": "Counterspell",
      "class": "MAGE",
      "type": "interrupt",
      "interruptible": false,
      "cooldownSec": 24
    },
    "51514": {
      "name": "Hex",
      "class": "SHAMAN",
      "type": "cc",
      "ccCategory": "incapacitate",
      "durationSec": 8,
      "breakOnDamage": true,
      "interruptible": false
    }
  },
  "enemySpells": {
    "dungeons": {
      "darkflame-cleft": {
        "453555": {
          "name": "Void Surge",
          "casterId": 12345,
          "casterName": "Darkwatcher",
          "interruptible": true,
          "castDurationSec": 2.5,
          "priority": "high",
          "notes": "Heavy damage to random target if not interrupted"
        },
        "453601": {
          "name": "Shadowquake",
          "casterId": 12345,
          "casterName": "Darkwatcher",
          "interruptible": false,
          "castDurationSec": 2.0,
          "priority": "avoid",
          "notes": "AoE ground slam - move out"
        }
      }
    }
  }
}
```



```

    }
  }
},
"defensives": {
  "PALADIN": {
    "RETRIBUTION": [
      { "spellId": 642, "name": "Divine Shield", "durationSec": 8, "cooldownSec": 8 },
      { "spellId": 184662, "name": "Shield of Vengeance", "durationSec": 15, "cooldownSec": 15 },
      { "spellId": 1022, "name": "Blessing of Protection", "durationSec": 10, "cooldownSec": 10 }
    ],
    "HOLY": [
      { "spellId": 642, "name": "Divine Shield", "durationSec": 8, "cooldownSec": 8 },
      { "spellId": 31821, "name": "Aura Mastery", "durationSec": 8, "cooldownSec": 8 }
    ]
  },
  "ROGUE": {
    "ASSASSINATION": [
      { "spellId": 31224, "name": "Cloak of Shadows", "durationSec": 5, "cooldownSec": 5 },
      { "spellId": 5277, "name": "Evasion", "durationSec": 10, "cooldownSec": 10 },
      { "spellId": 1966, "name": "Feint", "durationSec": 6, "cooldownSec": 6 }
    ]
  },
  "SHAMAN": {
    "ELEMENTAL": [
      { "spellId": 108271, "name": "Astral Shift", "durationSec": 8, "cooldownSec": 8 },
      { "spellId": 20608, "name": "Reincarnation", "durationSec": 0, "cooldownSec": 0 }
    ]
  },
  "DEMONHUNTER": {
    "VENGEANCE": [
      { "spellId": 198589, "name": "Blur", "durationSec": 10, "cooldownSec": 10 },
      { "spellId": 187827, "name": "Metamorphosis", "durationSec": 15, "cooldownSec": 15 },
      { "spellId": 204021, "name": "Fiery Brand", "durationSec": 8, "cooldownSec": 8 }
    ]
  },
  "PRIEST": {
    "HOLY": [
      { "spellId": 47585, "name": "Dispersion", "durationSec": 6, "cooldownSec": 6 },
      { "spellId": 33206, "name": "Pain Suppression", "durationSec": 8, "cooldownSec": 8 }
    ]
  }
},
"drCategories": {
  "stun": {
    "resetTimerSec": 18,
    "pvpImmunityAfter": 2,
    "pveImmunityAfter": 3,
    "spells": [408, 853, 1833]
  },
  "incapacitate": {
    "resetTimerSec": 18,
    "pvpImmunityAfter": 2,
    "pveImmunityAfter": 3,
    "spells": [51514, 118, 2094, 6770]
  }
}

```

```

    },
    "disorient": {
      "resetTimerSec": 18,
      "pvpImmunityAfter": 2,
      "pveImmunityAfter": 3,
      "spells": [8122, 9484]
    },
    "silence": {
      "resetTimerSec": 18,
      "pvpImmunityAfter": 2,
      "pveImmunityAfter": 3,
      "spells": [15487, 202933]
    }
  }
}

```

14. UI/UX Specification

Design Language

- **Theme:** Dark game UI — deep navy/black backgrounds, subtle borders, gold accent colour
- **Primary background:** #0a0c10
- **Panel background:** #0f1219
- **Card background:** #141820
- **Hover state:** #1a2030
- **Border:** #1e2535 (subtle), #252d40 (visible)
- **Accent gold:** #e8a820
- **Death/damage red:** #e84040
- **Interrupt/info blue:** #40b8e8
- **CC/magic purple:** #b040e8
- **Heal/good green:** #40e87a

Class colours (standard WoW):

- **Paladin:** #f58cba
- **Priest:** #ffffff
- **Rogue:** #fff569
- **Shaman:** #0070de
- **Demon Hunter:** #a330c9

Frame Dimensions

- Main frame: 960px wide × dynamic height (max ~650px, scrollable)
- Titlebar: 42px tall
- Tab bar: 38px tall
- Filter bar: 38px tall
- Content area: remaining space, internally scrollable per tab

Tab Layout

All tabs follow this structure:

1. **Encounter header** — dungeon name, meta (difficulty, pulls, duration), key stats for that tab
2. **Section header** — labelled divider with event count
3. **Content** — tab-specific content (cards, tables, timelines)

Interaction Model

- Single-click card headers to expand/collapse
- Hover on CC segments, cast bars, timeline events shows tooltip
- Filter chips are toggle buttons — click to include/exclude
- Preset dropdown saves/restores entire filter state
- `/c1` opens at last active tab
- Frame position remembered per character

Empty States

Every tab must handle these states gracefully:

- `CombatLedgerDB` is nil → "Companion app not detected. Install CombatLedger Companion to enable post-combat analysis."
- `CombatLedgerDB` exists but no sessions → "No sessions recorded yet. Run a dungeon or raid, then type `/reload`."
- Session exists but no deaths (on Deaths tab) → "No deaths in this session. Well played."
- Session exists but no interrupts (on Interrupts tab) → "No interruptible casts detected in this session."

15. Testing Strategy

Test Fixtures

Maintain a directory of real `WoWCombatLog.txt` excerpts in `tests/fixtures/`:

- `fixture_death_simple.txt` — single player death with clear causal chain
- `fixture_death_no_defensive.txt` — death where a defensive was available unused
- `fixture_interrupts_mixed.txt` — mix of successful and missed interrupts
- `fixture_cc_with_dr.txt` — CC chain demonstrating DR reduction
- `fixture_full_key.txt` — complete Mythic+ key, multiple pulls
- `fixture_wipe.txt` — encounter with no `ENCOUNTER_END` success (wipe)

Unit Tests (Vitest)

Each engine module should have unit tests against fixtures:

```
// tests/parser/death.test.ts
describe('DeathRecapEngine', () => {
  it('identifies fatal event correctly', async () => {
    const events = await parseFixture('fixture_death_simple.txt');
    const deaths = buildDeathRecaps(events);
    expect(deaths[0].fatalSpellName).toBe('Shadowquake');
    expect(deaths[0].overkill).toBe(12820);
  });

  it('identifies available unused defensive', async () => {
    const events = await parseFixture('fixture_death_no_defensive.txt');
    const deaths = buildDeathRecaps(events);
    const divineShield = deaths[0].availableDefensives.find(d => d.spellId === 642);
    expect(divineShield?.available).toBe(true);
    expect(divineShield?.wasUsed).toBe(false);
  });
});
```

Lua Addon Testing

- Use the WoW addon test harness (manual — load addon in WoW, verify no Lua errors via `/console scriptErrors 1`)
- Test SavedVariables round-trip: write test data via companion, verify addon reads it correctly
- Test empty state rendering
- Test filter chip interactions

Integration Test

End-to-end test: pipe a fixture file through the companion, verify the SavedVariables output, load into addon mock environment, verify data access.

16. Coding Standards & Conventions

TypeScript (Companion)

- Strict TypeScript (`"strict": true` in `tsconfig`)
- No `any` types — if uncertain, use `unknown` and `narrow`
- All parsed event types must be discriminated unions
- Parser functions are pure — no side effects, no global state
- Async/await throughout, no raw Promise chains
- Error handling: never `throw` from parser functions — return `Result<T, Error>` pattern

```
type Result<T, E = Error> = { ok: true; value: T } | { ok: false; error: E };
```

- File naming: `camelCase.ts` for modules, `PascalCase.ts` for classes
- All public interfaces exported from module index files

Rust (Companion Backend)

- Standard Rust formatting (`rustfmt`)
- All `unwrap()` calls must have a comment explaining why panic is acceptable or be replaced with proper error handling
- Use `anyhow` for error propagation in application code
- Use `thiserror` for library error types

Lua (Addon)

- Follow Ace3 conventions for addon structure
- Prefix all global functions and variables with `CL_` or access through the addon object
- All frame event handlers use `:` method syntax
- No global variable pollution — use local variables at module scope
- Comment all non-obvious API calls explaining what they do and any 12.0 restrictions that affected the approach

Commit Conventions

- `feat(companion): add CLEU parser for SPELL_DAMAGE events`
 - `feat(addon): implement Deaths tab UI`
 - `fix(parser): handle log file rotation correctly`
 - `data(spells): add Darkflame Cleft enemy spell metadata`
 - `test(death): add fixture for multi-mob death scenario`
-

17. Known Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
SavedVariables file size exceeds WoW limits	Medium	High	Cap at 20 sessions, trim oldest. Warn user in companion tray if output file > 1MB. Target max 500KB/session.
WoW install path auto-detection fails	Medium	Medium	Fallback to manual path picker in companion settings. Show clear error message.
DR category database has errors, producing wrong CC durations	High	Medium	Unit test every DR category against known log fixtures. Flag as "Beta" in UI. Community GitHub issues for corrections.
Companion app fails to write before player /reloads	Low	Low	Write on <code>ENCOUNTER_END</code> , not on zone change. File write is fast (<50ms for target data size).
Blizzard changes SavedVariables format or path in a patch	Low	High	Abstract path resolution behind a config layer. Version-gate the SavedVariables schema.
COMBATANT_INFO spec ID mapping becomes stale after patch	Medium	Low	Spec IDs are stable between minor patches. Update on major patch. Community PR pipeline for corrections.
Players distrust companion app (security concern)	High	High	Open source companion on GitHub. CurseForge distribution. No network requests. Document exactly what the app reads and writes. Privacy policy in README.
Tauri 2.x breaking changes during development	Low	Medium	Pin dependency versions. Read Tauri changelog before upgrading.

Quick Reference: Starting a Session

When beginning a new Claude Code session on this project, orient yourself with:

1. **What phase are we in?** Check which Phase tasks are complete vs pending
2. **What's the current data flow?** Companion → SavedVariables → Addon
3. **What can't the addon do?** It cannot read files, make requests, or process combat events — all logic is in TypeScript
4. **Where does the data come from?** `WoWCombatLog.txt` on disk, read by companion
5. **What's the communication bridge?** SavedVariables `.lua` file written by companion, read by addon on UI load

The golden rule: if you're adding logic to the Lua addon, stop and ask whether it belongs in the TypeScript companion instead.

End of CombatLedger Claude Code Project Brief